

02_Graph_Theory_Shortest_Path (Dijkstra, SPFA, Floyd)最短路_板子

Dijkstra 单源最短路（无向带权图）板子

无向图版本：

```
#include<bits/stdc++.h>
// #define int long long
using namespace std;
using ll=long long;
const ll inf=1ll<<60;

//这段代码实现了经典的 Dijkstra 单源最短路（无向带权图），
//用优先队列按距离扩展节点，最终输出从 s 到 t 的最短距离
//（如果不可达则输出 inf 的数值）

void work(){
    int n,m,s,t;
    cin>>n>>m>>s>>t;    s--;
    t--;
    vector<vector<pair<int,int> > > g(n);
    for(int i=0;i<m;i++){
        int u,v,w;
        cin>>u>>v>>w;
        u--;
        v--;
        g[u].emplace_back(v,w);
        g[v].emplace_back(u,w); //无向图
    }
    //仍用默认 max-heap，但把距离存成负数：q.emplace(-dist, node);    //这样 top 返回的负值最
    //大的也就是原始 dist 最小的条目
    priority_queue<pair<ll,int> > q;    //存点，边权
    vector<ll> dis(n,inf);    //到达每个点时的最短路
    dis[s]=0;    //起点为零
    q.emplace(0,s);
    while(size(q)){
        auto [d,u]=q.top();    //存 距离负值-dist，节点u
        q.pop();
        if(-d != dis[u]) continue;
        for(auto [v,w] : g[u]){
            if(dis[u]+w<dis[v]){    //如果s->u->v路径长度比原来要小，则更新，那么就加进来这条边
                dis[v]=dis[u]+w;
                q.emplace(-dis[v],v);
            }
        }
    }
    cout<<dis[t];    //到达终点的 最短路径
} signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    int t=1;
    //    cin>>t;
```

```

        while(t--){
            work();
        }
    }
}

```

```

/*
4 6 1
1 2 2
2 3 2
2 4 1
1 3 5
3 4 3
1 4 4

0 2 4 3
*/

```

有向图版本：

```

#include<bits/stdc++.h>
// #define int long long
using namespace std;
using ll=long long;
const int inf = (1ll << 31) - 1;

//这段代码实现了经典的 Dijkstra 单源最短路（无向带权图），
//用优先队列按距离扩展节点，最终输出从 s 到 t 的最短距离
//（如果不可达则输出 inf 的数值）

struct o{
    ll v, w;    //u->v 边权w
};

void work(){
    int n, m, s;
    cin >> n >> m >> s;
    vector<vector<o> > e(n+1);
    for(int i=0; i<m; i++){
        ll u, v, w;
        cin >> u >> v >> w;
        e[u].push_back({v, w}); //有向图存边
    }

    vector<int> dis(n+1, inf), vis(n+1, 0);
    dis[s] = 0;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    pq.push({0, s});
    while(!pq.empty()){
        auto nod = pq.top();
        auto [d,u] = nod;    //存距离，节点 按距离排序
        pq.pop();
        if(vis[u]) continue;
        vis[u]=1;
    }
}

```

```

        for(auto [v, w] : e[u]){
            if(dis[v] > dis[u]+w){ //如果s->u->v路径长度比原来要小，则更新，那么就加进来这条边
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }

    for(int i=1; i<=n; i++){
        cout << dis[i];
        if(i != n) cout << " ";
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T = 1;
    //    cin >> T;
    while(T--){
        work();
    }
}

/*
4 6 1
1 2 2
2 3 2
2 4 1
1 3 5
3 4 3
1 4 4

0 2 4 3
*/

```

Floyd 多源最短路 板子

[\(18条未读私信\) 代码查看](#)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll inf = 1ll<<60;

void work(){
    int n;
    cin >> n;
    vector<vector<ll>> dist(n, vector<ll>(n, inf));
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            int w;

```

```

        cin >> w;
        if(w == -1){
            if(i == j) dist[i][j] = 0;
            else dist[i][j] = inf;
        }
        else{
            dist[i][j] = w;
        }
    }
}

// Floyd-warshall
for(int k=0; k<n; k++){
    for(int i=0; i<n; i++){
        if(dist[i][k] == inf) continue; // 小优化: 如果 i->k 不可达, 跳过
        for(int j=0; j<n; j++){
            if(dist[k][j] == inf) continue;
            ll cand = dist[i][k] + dist[k][j]; //candidate
            if(cand < dist[i][j]) dist[i][j] = cand;
        }
    }
}

for(int i=0; i<n; i++){
    for(int j=0; j<n; j++){
        if(dist[i][j] >= inf/2) cout << -1;
        else cout << dist[i][j];
        if(j+1 < n) cout << " ";
    }
    cout << '\n';
}

}

signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    int T=1;
    //    cin>>T;
    while(T--){
        work();
    }
}

/*
4
0 1 -1 -1
-1 0 1 -1
-1 -1 0 1
1 -1 -1 0

0 1 2 3
3 0 1 2
2 3 0 1
1 2 3 0
*/

```

SPFA 判负环 板子

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18; // 极大值防溢出

struct Edge {
    int v;
    ll w;
};

// n: 节点数, s: 起点
// g: 邻接表存图 vector<Edge> g[N]
// dist: 最短路距离数组
// 返回值: 如果存在从起点可达的负环, 返回 true; 否则返回 false
bool spfa(int n, int s, const vector<vector<Edge>>& g, vector<ll>& dist) {
    dist.assign(n + 1, INF);
    vector<int> cnt(n + 1, 0); // cnt[i] 记录从起点 s 到 i 的最短路包含几条边
    vector<bool> in_q(n + 1, false); // in_q[i] 记录节点 i 当前是否在队列中
    queue<int> q;

    // 1. 初始化起点
    dist[s] = 0;
    q.push(s);
    in_q[s] = true;

    // 2. 队列 BFS 松弛
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        in_q[u] = false; // 出队后标记为不在队列中

        // 遍历所有从 u 出发的边
        for (const auto& edge : g[u]) {
            int v = edge.v;
            ll w = edge.w;

            // 尝试松弛
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                cnt[v] = cnt[u] + 1; // 经过的边数 + 1

                // 核心判负环逻辑: 抽屉原理!
                // 如果一条最短路经过了 >= n 条边 (即包含超过 n 个点), 必然存在负环!
                if (cnt[v] >= n) return true; // 发现负环, 立刻终止!

                // 如果 v 被更新了, 且 v 不在队列中, 就把它加进队列
                // (因为 v 变小了, 它有可能去更新它的邻居)
                if (!in_q[v]) {
                    q.push(v);
                    in_q[v] = true;
                }
            }
        }
    }
}
```

```

        }
    }
}
return false; // 队列空了也没触发负环，说明没有负环
}
/*

```

SPFA 在随机图上跑得飞快，复杂度近似 $O(k \cdot E)$ (k 是一个小常数)。

但是！只要出题人稍微有点良心（或者坏心眼），他可以故意构造一种“菊花图”或“网格图”，把 **SPFA** 诱导进队列反复横跳，它的复杂度会瞬间退化到最原始的 $O(V \times E)$ ，导致必定超时（**TLE**）！

只要图里没有负权边：死都不要用 **SPFA**！必须用 **Dijkstra**！

如果图里有负权边，或者题目明确要求判断负环：你没有选择，只能硬着头皮用这套 **SPFA** 板子，并祈祷出题人没有卡你的数据。

```

*/

```